

SMART CITY COMPLAINT MANAGEMENT PLATFORM

Enterprise Technical Architecture & Production Stack Specifications

1. Executive Summary

This specification proposes a scalable, secure, and robust technology stack designed to support municipal-level operations. To guarantee accountability and transparency, a modern complaint platform must extend beyond simple ticketing. It requires real-time routing, role-based workflows, GIS tracking, service level agreement (SLA) checkers, and data analytics dashboards. This report details technology stacks structured by stage—from a lightweight Minimum Viable Product (MVP) to a fully redundant high-availability government deployment.

2. Mandatory Municipal Modules

To satisfy government tenders and secure commercial deployments, the platform implements these essential modules mapped by priority level:

Module Name	Target Audience / Role	Priority Level	Platform Feature Description
Citizen Portal	Citizens (Web/Mobile)	Must-Have (5/5)	Filing, status tracker, AI category detection, chatbot support.
Admin Panel	Super Admin / IT Dept	Must-Have (5/5)	User database, system logs, blockchain audits, department configuration.
Officer App	Field Municipal Officers	Must-Have (5/5)	Assigned task grids, image upload (before/after), resolution remarks.
Department Dashboard	Department Heads	Must-Have (5/5)	Workload distribution, priority updates, duplicate rejection, ticket merge.
Commissioner Dashboard	City Commissioners / Mayor	High-Priority (5/5)	High-level performance trends, satisfaction scores, ward rankings.
GIS/Map Integration	All Users / Dispatchers	Must-Have (5/5)	Circle heat map overlays, coordinate plots, Leaflet open-street tiles.
SLA Management	Automation Engine	Must-Have (5/5)	Time-limit breach flags (12h to 7d limits), auto notifications & red alerts.
Escalation Workflow	Automation Engine	Must-Have (5/5)	Chain escalation (Officer -> Senior -> Commissioner -> Mayor).
Notifications	All Stakeholders	Must-Have (5/5)	Real-time alerts via Email, SMS gateway, and WhatsApp integrations.
Analytics & Reports	Management / Board	Must-Have (5/5)	PDF/CSV report generation, monthly trends charts, leaderboards.
AI Features	Automation / System	Medium-High (4/5)	Auto classification of description text, NLP sentiment analysis.
Asset Management	Field Teams	Medium-High (4/5)	Mapping municipal assets (lights, pipelines) to speed up repairs.
Transparency Ledger	General Public	Medium-High (4/5)	Blockchain-based write-only audit trail of complaint history.
QR-based Complaints	Citizens	Medium-High (4/5)	QR codes pasted on physical assets for quick category filing.
IoT Integration	Smart Devices	Optional (3/5)	Automatic complaint logging from smart streetlights and water sensors.

3. Technology Stack Options

Depending on the operational stage, budget constraints, and development timelines, three distinct architectural pathways are detailed below.

Option 1: Minimum Viable Product (MVP) / Demo Scale

Optimized for 1 developer, targeting quick prototyping (2-4 months) and government live demos with zero overkill components.

Layer	Technology Selection	Strategic Rationale
Frontend	React.js, TypeScript, Tailwind CSS	Rapid component construction and compile-time type-safety.
Backend	Spring Boot (Java 21)	Industry standard for municipal software, facilitating easy transition to prod.
Database	PostgreSQL	Highly reliable relational database with built-in text search capabilities.
Storage	MinIO (Local S3-Compatible)	Simulated S3 object storage running locally to keep hosting costs at zero.
Auth	JWT + Simulated OTP	Standard stateless authentication without external SMS gateway fees.
Maps	Leaflet.js + OpenStreetMap	Zero commercial API key subscription overheads, fully responsive maps.
API Docs	Swagger / Springdoc OpenAPI	Interactive web UI testbed automatically synchronized with backend code.
Deployment	Docker Compose	Allows running the entire stack on a single Virtual Private Server (VPS).

Option 2: Production & Government-Grade Ready

Designed for institutional tenders, strict security audits, and deployment in active municipalities.

Layer	Technology Selection	Strategic Rationale
Frontend	React.js, TS, Tailwind, Redux Toolkit	Advanced state management for real-time dashboards.
Backend	Spring Boot (Java 21) or NestJS	Multi-threaded high-throughput framework with clean modular layout.
Database	PostgreSQL + PostGIS Extension	Enables geospatial ward polygon geometry queries (location-in-ward check).
Cache	Redis	Sub-millisecond query result caching and shared cluster session storage.
Storage	AWS S3 or equivalent Object Store	Durable, replicated cloud bucket for user upload evidence images.
Auth	JWT + OAuth2 + Multi-Factor OTP	Rigorous secure authentication supporting single sign-on (SSO).
Notifications	SMS Gateway, Email, WhatsApp APIs	Omnichannel alerts triggered instantly on citizen complaint status edits.
Search	PostgreSQL Full-Text Search	High-performance text indexing without the operational cost of Elasticsearch.
Monitoring	Prometheus + Grafana	Visual dashboards representing server load, database connections, and API latency.
Deployment	Kubernetes (K8s)	Automated scaling, self-healing node clusters, and rolling updates.

Option 3: Enterprise Plus (High Availability & Multi-Tenant scale)

Engineered for smart cities with populations exceeding 1 million, managing high traffic spikes and vast datasets.

Layer	Technology Selection	Strategic Rationale
API Gateway	Spring Cloud Gateway	Central entry point handling routing, SSL termination, and rate-limiting.
Reverse Proxy	Nginx	Frontend asset hosting and upstream load balancing.
Message Queue	Apache Kafka / RabbitMQ	Asynchronous task worker distribution (e.g. background SLA/escalation checks).
Logging	ELK / EFK Stack	Centralized storage and indexing of distributed microservice logs.
Secrets	HashiCorp Vault	Secure, audited database credentials and API keys storage.
Backups / CDN	PostgreSQL Replication + Cloudflare	Hot-standby replicas and edge network caching with DDoS protection.
Search Engine	Elasticsearch	Used once database indexes cross 10 million complaints for complex facets.

4. Strategic Architectural Recommendations & Guidelines

Backend Language Choice (Java vs. NestJS):

Java with Spring Boot is highly recommended for government portals. Public sector bids and departments preferentially trust Java due to legacy libraries and long-term support. NestJS (TypeScript) should only be selected if the development team lacks Java experience.

Database and GIS Selection (PostgreSQL + PostGIS vs. MySQL):

PostgreSQL is the undisputed database choice. The PostGIS extension offers true geographic queries, allowing polygon containment checks to verify if coordinates fall within specific ward boundaries. MySQL lacks similar spatial index efficiency.

Map Provider selection (OpenStreetMap + Leaflet vs. Google Maps):

Municipalities heavily dislike recurring Google Maps API usage bills. The combination of OpenStreetMap tiles and Leaflet.js is completely free, open-source, and does not require credit card configurations, while remaining highly performant. Google Maps should only be used as a configurable client-side option.

Search Optimization (PostgreSQL FTS vs. Elasticsearch):

Do not introduce Elasticsearch in the early stages. PostgreSQL Full-Text Search is capable of handling up to 100,000 complaints with custom dictionary indexing. Introduce Elasticsearch only when complaint history surpasses millions of records to avoid database CPU spikes.

Container Orchestration (Docker Compose vs. Kubernetes):

Docker Compose is sufficient for single VPS hosting, minimizing devops configuration. Transition to Kubernetes (K8s) only for multi-server node redundancy and automatic cluster failover in high-load setups.